1. Explain the architecture of Java. Discuss JDK, JRE, and JVM in detail with a diagram.

Java architecture is designed to achieve platform independence.

It consists of three main components: JDK, JRE, and JVM.

JDK (Java Development Kit): It is a complete development package that includes tools like compiler (javac), debugger, and libraries required to develop Java programs.

JRE (Java Runtime Environment): It provides libraries and JVM required to run Java programs.

JVM (Java Virtual Machine): It executes bytecode and converts it into machine code. It also manages memory and security.

Diagram Flow: Source Code → Compiler → Bytecode → JVM → Output

2. Write a detailed note on the differences between Primitive and Non-Primitive data types in Java.

Primitive Data Types: int, float, char, boolean etc. They store actual values and are stored in stack memory.

Non-Primitive Data Types: Arrays, Strings, Classes. They store references and are stored in heap memory.

Differences:

1. Primitive holds value, Non-primitive holds reference.

2. Primitive is faster, Non-primitive is slower.

3. Primitive has fixed size, Non-primitive size can vary.

3. Explain the "Object Oriented" concepts: Encapsulation, Inheritance, and Polymorphism with real-world examples.

Encapsulation: Wrapping data and methods together. Example: Bank account with private balance.

Inheritance: One class inherits another. Example: Car inherits Vehicle.

Polymorphism: One method with many forms. Example: same function behaves differently.

4. What is a Constructor? Explain the different types of constructors with a Java program demonstrating Constructor

Chaining.

Constructor is a special method used to initialize objects.

Types: Default constructor and Parameterized constructor.

Constructor chaining is calling one constructor from another using 'this()'.

```
class A {
 A(){ this(10); System.out.println("Default Constructor"); }
 A(int x){ System.out.println("Parameterized Constructor: "+x); }
 public static void main(String[] args){ new A(); }
}
```

5. Differentiate between Method Overloading and Method Overriding. Provide examples for each.

Overloading: Same method name, different parameters (compile-time).

Overriding: Same method in subclass (runtime).

```
class A { void show(){ } void show(int a){ } }
class B extends A { void show(){ } }
```

6. Write a Java program to demonstrate the use of the this keyword.

this keyword refers to current object.

```
class Test {
 int x;
 Test(int x){ this.x = x; }
 void display(){ System.out.println(x); }
 public static void main(String[] args){ Test t = new Test(10); t.display(); }
}
```

7. Explain the concept of Garbage Collection. How can you make an object eligible for Garbage Collection? Write a program using finalize().

Garbage Collection automatically deletes unused objects.

An object becomes eligible when no reference points to it.

```
class GC {
 protected void finalize(){ System.out.println("Object destroyed"); }
 public static void main(String[] args){ GC obj = new GC(); obj = null; System.gc(); }
}
```

8. Write a Java program to check if a number is Prime or Not.

```
class Prime {
 public static void main(String[] args){
 int n=7, flag=0;
 for(int i=2;i<=n/2;i++){
 if(n%i==0){ flag=1; break; }
 }
 if(flag==0) System.out.println("Prime");
 else System.out.println("Not Prime");
 }
}
```

9. Explain Call by Value and Call by Reference in the context of Java. Is Java strictly Call by Value? Justify.

Java is strictly call by value.

It passes copy of variables. For objects, reference copy is passed.

```
class Test {
 void change(int x){ x=50; }
 public static void main(String[] args){ int x=10; new Test().change(x); System.out.println(x); }
}
```